

HomeScript: Uma DSL para Automação Residencial Baseada em Compilador

Mateus Hypolito Silva

Fundação Herminio Ometto — FHO Araras

Engenharia da Computação — Teoria da Computação e Compiladores

Resumo

Este artigo apresenta a HomeScript, uma Linguagem de Domínio Específico (DSL) para automação residencial cujos comandos são escritos em português. O compilador desenvolvido aplica as quatro etapas clássicas de tradução — análise léxica com expressões regulares e autômatos finitos, análise sintática com gramática BNF e geração de AST, análise semântica com sete categorias de verificação, e geração de código Python executável. A linguagem foi validada por meio de uma rotina diária completa e demonstra integração com hardware real via GPIO do Raspberry Pi. Os resultados confirmam a viabilidade de DSLs como interface de programação acessível para domínios específicos.

Palavras-chave — DSL, compilador, automação residencial, PLY, Raspberry Pi, autômatos finitos, gramática formal.

I. INTRODUÇÃO

A crescente adoção de dispositivos conectados em residências tem impulsionado o desenvolvimento de soluções de automação residencial. No entanto, a programação direta desses dispositivos exige conhecimento técnico avançado, o que limita o acesso de usuários não especializados.

Este trabalho propõe a HomeScript, uma Linguagem de Domínio Específico (DSL) para automação residencial, cujos comandos são escritos em português e mapeados para ações concretas sobre dispositivos de uma casa inteligente. O compilador desenvolvido aplica as etapas clássicas de tradução — análise léxica, sintática, semântica e geração de código — e produz código Python executável como saída.

A motivação central é demonstrar, de forma prática, a aplicação de conceitos de autômatos finitos, gramáticas formais e compiladores em um problema de software real.

II. REFERENCIAL TEÓRICO

A. Linguagens de Domínio Específico

DSLs são linguagens projetadas para um domínio restrito, oferecendo expressividade adequada com menor complexidade que linguagens de propósito geral [1]. No contexto de automação residencial, uma DSL permite que o usuário descreva rotinas sem conhecer detalhes de implementação.

B. Autômatos e Expressões Regulares

A análise léxica é fundamentada na teoria de autômatos finitos. Cada token da linguagem é definido por uma expressão regular equivalente a um Autômato Finito Determinístico (AFD) [2]. O token DISPOSITIVO, por exemplo, segue o padrão

C. Análise Semântica

A análise semântica percorre a AST antes da execução e verifica sete classes de erros que a gramática não consegue capturar:

AS-01: Dispositivo não declarado na tabela de símbolos;

AS-02: Operação incompatível com o tipo do dispositivo;

AS-03: Escrita em sensor de somente leitura;

AS-04: Valor numérico fora do intervalo válido;

AS-05: Comparação de tipos incompatíveis em condições;

AS-06: Referência a cena não definida;

AS-07: Número de repetições inválido (≤ 0).

O analisador acumula todos os erros antes de encerrar, seguindo o padrão de compiladores reais.

D. Geração de Código

O compilador percorre a AST nó a nó e gera código Python equivalente. Estruturas de controle da HomeScript são mapeadas diretamente para construções Python: `se/entao/fim` torna-se `if/else`, `repetir N vezes` torna-se `for _ in range(N)`. O arquivo `.py` gerado é autossuficiente e não requer o compilador para execução.

V. ROTINA DIÁRIA — CASO DE USO

Para validar a expressividade da linguagem, foi desenvolvida uma rotina diária completa cobrindo os períodos da manhã, tarde e noite. A rotina exercita todas as construções da HomeScript.

A. Rotina da Manhã

ambiente.componente, reconhecido por um AFD de quatro estados.

C. Gramáticas Livres de Contexto

A análise sintática utiliza gramáticas livres de contexto (GLC) para definir a estrutura da linguagem. A Backus-Naur Form (BNF) formaliza as produções e o parser verifica se o programa obedece a essa gramática, gerando uma Árvore Sintática Abstrata (AST) [3].

III. A LINGUAGEM HOMEScript

A. Visão Geral

A HomeScript possui sintaxe em português, com comandos que mapeiam diretamente para ações sobre dispositivos residenciais. Dispositivos são referenciados pelo padrão ambiente.componente (ex.: sala.luz, quarto.ar_condicionado).

B. Principais Construções

A linguagem suporta seis construções principais:

- (1) Comandos de ação: ligar, desligar e ajustar;
- (2) Temporização: aguardar N;
- (3) Condicional: se/entao/senao/fim;
- (4) Repetição: repetir N vezes/fim;
- (5) Cenas predefinidas: cena "nome";
- (6) Condição de horário: se hora > HH:MM.

Exemplo de programa HomeScript:

```
ligar sala.luz
ajustar quarto.ar_condicionado para 22
se sala.temperatura > 24 entao
    ligar sala.ventilador
senao
    desligar sala.ventilador
fim
cena "dormir"
```

IV. ARQUITETURA DO COMPILADOR

A. Análise Léxica

O analisador léxico foi implementado em Python com a biblioteca PLY (Python Lex-Yacc). Cada token é definido por uma expressão regular. Tokens de maior prioridade são implementados como funções Python, garantindo precedência correta sobre padrões mais gerais.

A Tabela I apresenta os principais tokens da linguagem.

TABELA I. TOKENS DA LINGUAGEM HOMEScript

Token	Padrão (Regex)	Exemplo
DISPOSITIVO	[a-z][a-z_]*\.[a-z][a-z_]+	sala.luz

A rotina matinal aciona a cena "acordar", liga as luzes dos ambientes principais, a cafeteira e o som ambiente. Em seguida, verifica a temperatura do quarto e ajusta o ar-condicionado conforme necessário:

```
cena "acordar"
ligar sala.luz
ligar cozinha.cafeteira
ligar sala.som_ambiente
se quarto.temperatura < 18 entao
    ajustar quarto.ar_condicionado para 22
senao
    desligar quarto.ar_condicionado
fim
```

B. Rotina da Tarde

Durante a ausência, todos os dispositivos são desligados para economia de energia. Ao retorno, os ambientes são reativados com temperatura controlada.

C. Rotina da Noite

À noite, a cena "jantar" é ativada condicionalmente ao horário. Após o jantar, dispositivos são desligados progressivamente e a cena "dormir" encerra a rotina.

VI. INTEGRAÇÃO COM RASPBERRY PI

A HomeScript foi projetada para execução em um Raspberry Pi, onde o código Python gerado pelo compilador controla pinos GPIO para acionar módulos relé conectados a dispositivos reais.

A. Arquitetura de Hardware

Os pinos GPIO do Raspberry Pi operam em 3.3V, insuficientes para acionar dispositivos de 110/220V diretamente. O módulo relé atua como interruptor eletromecânico, isolando a tensão de controle da tensão de carga [4].

O mapeamento entre dispositivos da linguagem e pinos GPIO é definido em um dicionário Python, permitindo reconfiguração sem alteração da linguagem.

B. Modos de Interação

O usuário pode interagir com o sistema de três formas: (1) acesso direto via teclado e monitor; (2) acesso remoto via SSH; (3) execução automática via agendamento cron, eliminando interação manual para rotinas fixas.

VII. RESULTADOS E DISCUSSÃO

O compilador HomeScript foi implementado com sucesso, cobrindo todas as quatro etapas de tradução. A linguagem mostrou-se expressiva para o domínio de automação residencial, permitindo a descrição de

NUMERO	\d+(\.\d+)?	22, 28.5
PERCENTUAL	\d+(\.\d+)?%	75%
HORARIO	\d{2}:\d{2}	18:30
Reservadas	Dicionário fixo	ligar, se, fim

B. Análise Sintática

O parser foi implementado com PLY yacc, utilizando uma gramática livre de contexto definida em BNF. O resultado é uma AST composta por objetos da classe No, com atributos tipo, valor e filhos. As principais produções da gramática são:

```

<programa> ::= <lista_cmds>
<cmd>      ::= LIGAR DISPOSITIVO
           | DESLIGAR DISPOSITIVO
           | AJUSTAR DISPOSITIVO
PARA NUMERO
           | SE <cond> ENTAO
NEWLINE
           <lista_cmds> FIM
           | REPETIR NUMERO VEZES
           NEWLINE <lista_cmds>
FIM
<cond>    ::= DISPOSITIVO <op>
<valor>   ::= DISPOSITIVO <op>
           | HORA <op> HORARIO
           | <cond> E <cond>
           | <cond> OU <cond>

```

rotinas complexas com sintaxe simples e legível.

A geração de código Python como saída elimina dependências de runtime: o arquivo gerado executa em qualquer ambiente Python sem necessidade do compilador. No Raspberry Pi, a integração GPIO permite acionamento de dispositivos reais sem alteração do código-fonte HomeScript.

A análise semântica demonstrou eficácia na detecção de erros lógicos antes da execução, evitando falhas em tempo de execução que poderiam danificar dispositivos físicos.

VIII. CONCLUSÃO

Este trabalho demonstrou a aplicação prática de autômatos finitos, gramáticas formais e técnicas de compilação no desenvolvimento de uma DSL para automação residencial. A HomeScript alcança seu objetivo de tornar a programação de dispositivos acessível a usuários não técnicos, sem abrir mão do rigor formal exigido pela disciplina.

Como trabalhos futuros, destacam-se: suporte a variáveis e expressões aritméticas, análise semântica com inferência de tipo, comunicação MQTT para integração com plataformas IoT e interface gráfica para edição e depuração de programas HomeScript.

. REFERÊNCIAS

- [1] M. Fowler, Domain-Specific Languages. Addison-Wesley, 2010.
- [2] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, Compilers: Principles, Techniques, and Tools. 2nd ed. Pearson, 2006.
- [3] J. E. Hopcroft, R. Motwani, J. D. Ullman, Introduction to Automata Theory, Languages, and Computation. 3rd ed. Pearson, 2006.
- [4] Raspberry Pi Foundation, Raspberry Pi Hardware Documentation. Disponível em: <https://www.raspberrypi.com/documentation/>. Acesso em: maio 2026.
- [5] M. H. Silva, HomeScript — Repositório do Projeto. GitHub, 2026. Disponível em: <https://github.com/Mateus140901>. Acesso em: maio 2026.